LLM Controlled Generation

Contents

1 Part 1: Structured Generation [8 points] 3
1.1 Overview . . . 3
1.2 Module 1 — Naive vs. DFA Masking (`deterministic_finite_automaton.py`) . . . 3
1.2.1 Motivation . . . 3
1.2.2 Algorithm: Naive Masking . . . 3
1.2.3 Algorithm: DFA-Based Masking . . . 3
1.2.4 What to implement . . . 4
1.2.5 Running the benchmark . . . 4
1.3 Module 2 — Character-level DFA to Token-level FSM (`fsm_token.py`) . . . 4
1.3.1 Motivation . . . 4
1.3.2 Algorithm . . . 4
1.3.3 The `_walk_token_through_fsm` function . . . 4
1.3.4 What to implement . . . 5
1.3.5 Running the demo . . . 5
1.4 Module 3 — Coalescence: Longest-Token Optimization (`coalescence.py`) . . . 5
1.4.1 Motivation . . . 5
1.4.2 Algorithm: `build_tokenizer_index` . . . 5
1.4.3 What to implement . . . 5
1.4.4 Running the benchmark . . . 6
1.4.5 JSON regex patterns . . . 6
1.5 Module 4 — End-to-End Constrained Generation (`picoGPT_generate.py`) . . . 6
1.5.1 Motivation . . . 6
1.5.2 picoGPT: GPT-2 in NumPy . . . 6
1.5.3 Generation modes . . . 6
1.5.4 What to implement . . . 6
1.5.5 Running the demo . . . 7
1.6 Summary: The Full Pipeline . . . 7

2 Part 2: Meta-Generation [8 points] 8
2.1 Overview . . . 8
2.2 The Problem: Best-of-N Selection . . . 8
2.3 Baseline: pass@1 . . . 8
2.4 Strategy 1: Best-of-N by Log-Probability (`rank_by_logprob`) . . . 9
2.4.1 Idea . . . 9
2.4.2 Algorithm . . . 9
2.4.3 Properties . . . 9
2.4.4 What to implement . . . 9
2.5 Strategy 2: MBR with Edit Similarity (`rank_by_edit_sim`) . . . 9
2.5.1 Idea . . . 9
2.5.2 Algorithm . . . 9
2.5.3 Properties . . . 10
2.5.4 What to implement . . . 10

2.6 Strategy 3: MBR with Execution Similarity (`rank_by_exec_sim`) . . . 10
2.6.1 Idea . . . 10
2.6.2 Algorithm . . . 10
2.6.3 Properties . . . 10
2.6.4 What to implement . . . 10
2.7 Evaluation (`evaluate_problem`) . . . 10
2.7.1 What to implement . . . 11
2.8 Utility Module (`utils.py`) . . . 11
2.9 Running the Evaluation . . . 11
2.9.1 Prerequisites . . . 11
2.9.2 Running . . . 11
2.9.3 Command-line options . . . 11
2.9.4 Example output . . . 12
2.10 Summary . . . 12

3 Part 3: Self-Correction [8 points] 13
3.1 Overview . . . 13
3.2 Context: Formal Verification with Verus . . . 13
3.2.1 Example specification (`examples/incr_list.rs`) . . . 13
3.3 Stage 1: Parallel Initial Generation (`parallel_generator`) . . . 13
3.3.1 Idea . . . 13
3.3.2 Algorithm . . . 14
3.3.3 What to implement . . . 14
3.4 Stage 2: Iterative Refinement with Tree Search . . . 14
3.4.1 Idea . . . 14
3.4.2 The Refinement Loop . . . 14
3.4.3 What to implement . . . 15
3.5 Code Validation (`_validate_code`) . . . 15
3.6 Trivial Proof Detection (`check_pairs`, `check_pairs_loop`) . . . 15
3.7 Scoring Function (`evaluate_code`) . . . 16
3.8 System Prompt . . . 16
3.9 Running the Pipeline . . . 16
3.9.1 Prerequisites . . . 16
3.9.2 Running . . . 16
3.9.3 Command-line options . . . 16
3.9.4 Bonus . . . 17
3.10 Summary: The Full Pipeline . . . 17

1 Part 1: Structured Generation [8 points]

LLMs generate text token by token, sampling from the full vocabulary at each step. But many applications require the output to follow a strict format: valid JSON, a specific schema, a phone number, etc. **Structured generation** guarantees that every generated token is part of a valid output by constraining the sampling at each step.

This part introduces a progression of techniques, from a naive baseline to a fully optimized constrained decoding pipeline. Each module builds on the previous one.

1.1 Overview

Module	Description
`deterministic_finite_automaton.py`	Naive regex masking vs. DFA-based masking
`fsm_token.py`	From character-level DFA to token-level FSM
`coalescence.py`	Longest-token coalescence
`picoGPT_generate.py`	End-to-end constrained generation with GPT-2
`utils.py`	GPT-2 weight loading and caching (utility, no exercises)

1.2 Module 1 — Naive vs. DFA Masking (`deterministic_finite_automaton.py`)

1.2.1 Motivation

Given a regex pattern (e.g. `([0-9]+)?\.[0-9]+` for decimal numbers), how do we ensure the LLM only generates matching strings?

The simplest idea: at each generation step, try appending every token in the vocabulary to the current completion and check whether the result could still lead to a valid match. This is the **naive approach**.

1.2.2 Algorithm: Naive Masking

For each generation step:

1. For each token `t` in the vocabulary of size `V`:
   - Form the tentative string `completion + t`
   - Run a **partial** regex match: does this prefix still have a chance of completing to a full match?
   - If not, mask the token (set its logit to `-inf`)
2. Sample from the masked distribution

This works but is **O(V)** regex evaluations per step, which is expensive for real vocabularies (GPT-2 has 50,257 tokens).

1.2.3 Algorithm: DFA-Based Masking

A regex can be compiled into a **Deterministic Finite Automaton (DFA)**. A DFA has a finite set of states and deterministic transitions on each input character. Instead of re-running the regex from scratch at every step, we can:

1. **Precomputation (one-time):** Parse the regex into a DFA. For each DFA state and each token in the vocabulary, walk the token's characters through the DFA and record where it lands. This produces a mapping: `(state, token_id) -> next_state`.
2. **Generation (per step):** Look up the current DFA state, retrieve the set of valid tokens, build the mask. This is an **O(1)** lookup.

The precomputation is `O(|states| x V x avg_token_length)` but is done only once. After that, each generation step is essentially free.

1.2.4 What to implement

- `naive_mask(completion, vocabulary, pattern)`: build a logit mask by trying every token with partial regex matching.
- `build_dfa_index(vocabulary, pattern)`: precompute the DFA transition table indexed by state and token.
- `dfa_mask(state, states_to_vocab, vocab_size)`: build a logit mask from the precomputed index in O(1).

1.2.5 Running the benchmark

```
uv run python structured_generation/deterministic_finite_automaton.py
```

This benchmarks both approaches across vocabulary sizes from 100 to 10,000 and prints a comparison table showing the speedup of DFA masking over naive masking.

1.3 Module 2 — Character-level DFA to Token-level FSM (`fsm_token.py`)

1.3.1 Motivation

Module 1 builds a DFA index, but the DFA itself operates on **characters**, while LLMs generate **tokens** (which can be multi-character strings like `"ing"`, `".2"`, or `"123"`). We need to bridge this gap by compiling the character-level DFA into a **token-level Finite State Machine (FSM)**.

1.3.2 Algorithm

The pipeline has three stages:

Stage 1 — Parse the regex into a character-level DFA. We use the `interegular` library which converts a regex pattern into a DFA with states, transitions, and an alphabet mapping characters to symbol indices.

Stage 2 — Clean up the DFA (`make_deterministic_fsm`). The raw DFA from `interegular` may contain:

- An implicit "oblivion" (dead/sink) state
- Non-contiguous state numbering
- The special `anything_else` sentinel for characters not explicitly in the alphabet

This function normalizes the DFA: collects all reachable states, remaps them to contiguous integers (0, 1, 2, ...), and builds a clean FSM object.

Stage 3 — Build the token-level FSM (`create_fsm_index_tokenizer`). For every (state, token) pair:

1. Walk the token's characters through the character-level DFA starting from that state (using `_walk_token_through_fsm`)
2. If the walk succeeds (no dead-end), record the transition: `state --[token_id]--> landing_state`

The result is a `TokenFSM` object whose transitions are over token IDs instead of characters.

1.3.3 The `_walk_token_through_fsm` function

This is the core primitive. Given a DFA state and a multi-character token string:

- For each character, look up the symbol index in the DFA alphabet (handling the `anything_else` sentinel)
- Attempt the transition; if any character fails, return `None` (token rejected)

- If all characters succeed, return the final landing state

1.3.4 What to implement

- `make_deterministic_fsm(fsm)`: normalize a raw DFA to clean integer states.
- `_walk_token_through_fsm(fsm, state, token)`: walk a multi-character token through the character-level DFA.
- `create_fsm_index_tokenizer(fsm, tokenizer)`: build the full token-level FSM by iterating over all (state, token) pairs.

1.3.5 Running the demo

```
uv run python structured_generation/fsm_token.py
```

This demonstrates the full pipeline on the regex `([0-9]+)?\.[0-9]+` with a small vocabulary `["a", ".", ".2", "1"]`, showing the DFA states, the token-level transitions, and a constrained generation run.

1.4 Module 3 — Coalescence: Longest-Token Optimization (`coalescence.py`)

1.4.1 Motivation

The token-level FSM from Module 2 gives us an `O(1)` mask lookup per step, but we can go further.

Consider generating the string `"name"`. With a typical tokenizer vocabulary, there are many ways to produce it: `["name"]`, `["n", "ame"]`, `["na", "me"]`, `["n", "a", "m", "e"]`, etc. When the character-level DFA has a **deterministic path** forward (only one valid character transition at each step), all these tokenizations follow the same forced sequence. Instead of stepping through short tokens one at a time (each requiring an LLM call), we pick the **longest token** that covers as much of the deterministic path as possible. This skips the LLM forward pass entirely for those states and jumps as far ahead as possible in a single step.

1.4.2 Algorithm: `build_tokenizer_index`

1. Parse the regex and build the character-level DFA (reusing Module 2)
2. Build the token-level FSM index (same as Module 2)
3. **Precompute logit masks:** for each state, build a NumPy array of size `V` with `-inf` at disallowed positions and `0` at allowed positions.
4. **Longest-token coalescence:** for each state, check whether the character-level DFA has a deterministic path forward (using `_walk_deterministic`). If so, among all valid tokens from that state, pick the longest one that stays on the deterministic path and record it in a `coalesced_tokens` dictionary along with its landing state.

Returns:

- `TokenFSM`: the token-level FSM
- `precomputed_masks`: `dict[state] -> np.array` (one mask per state)
- `coalesced_tokens`: `dict[state] -> (token_id, landing_state)`
- `build_ms`: build time in milliseconds

1.4.3 What to implement

- `_walk_deterministic(fsm, state, n_chars)`: check that the character-level DFA is deterministic for `n_chars` steps from a given state.
- `build_tokenizer_index(pattern, tokenizer, vocabulary)`: the full pipeline including mask precomputation and longest-token coalescence.
- The generation loop that emits the longest coalesced token on deterministic paths, skipping the LLM entirely.

1.4.4 Running the benchmark

```
uv run python structured_generation/coalescence.py
```

This benchmarks on the real GPT-2 vocabulary (50,257 tokens) with three JSON regex patterns of increasing complexity. It compares the per-step time of FSM masking (without coalescence) vs. longest-token coalescence.

1.4.5 JSON regex patterns

The module defines three JSON patterns used for benchmarking:

- `JSON_SIMPLE`: a JSON object with two fixed fields (`name`, `age`) and a few allowed values
- `JSON_COMPLEX`: five fields with richer value ranges (names, ages 18-65, cities, booleans, scores)
- `JSON_VERY_COMPLEX`: deeply nested JSON with addresses, email validation, arrays, and many allowed values

---

1.5 Module 4 — End-to-End Constrained Generation (`picoGPT_generate.py`)

1.5.1 Motivation

Modules 1-3 build the masking infrastructure. This module puts it all together with an actual language model: a pure-NumPy implementation of GPT-2 (based on picoGPT by Jay Mody).

1.5.2 picoGPT: GPT-2 in NumPy

The module implements the full GPT-2 forward pass in pure NumPy:

- `gelu`, `softmax`, `layer_norm`, `linear`: basic building blocks
- `ffn`: feed-forward network (two linear layers with GELU)
- `attention`: scaled dot-product attention with causal mask
- `mha`: multi-head attention (split heads, attend, concatenate)
- `transformer_block`: layer norm + multi-head attention + layer norm + FFN (with residual connections)
- `gpt2`: token embeddings + position embeddings + transformer blocks + final layer norm + output projection

1.5.3 Generation modes

Unconstrained generation (`generate_unconstrained`):

- Standard autoregressive sampling with optional temperature
- Optional brace-depth tracking to stop after the closing `}` in JSON output
- No guarantee that the output is valid

Constrained generation (`generate_constrained`):

1. Build the token-level FSM from the regex pattern (using `build_tokenizer_index` from Module 3)
2. At each step:
   - If the current FSM state has a coalesced token: emit the longest token directly, **skip the forward pass**, and jump to the landing state
   - Otherwise: run the GPT-2 forward pass, apply the precomputed mask, sample
3. Advance the FSM state
4. Stop when the FSM reaches a final state with no outgoing transitions

The output is **guaranteed** to match the regex pattern.

1.5.4 What to implement

- `generate_unconstrained(...)`: autoregressive generation loop with optional brace-depth stopping.

- `generate_constrained(...)`: generation loop with FSM masking and longest-token coalescence.

1.5.5 Running the demo

```
uv run python structured_generation/picoGPT_generate.py
```

This loads GPT-2, encodes a prompt asking to convert user profiles to JSON, and runs:

1. Unconstrained generation (likely produces invalid JSON)
2. Constrained generation with `JSON_VERY_COMPLEX` (guaranteed valid output)

Both outputs are tested against the regex to verify correctness.

1.6 Summary: The Full Pipeline

```
Regex pattern
    |
    v
Character-level DFA          (interegular)
    |
    v
Clean DFA                    (make_deterministic_fsm)
    |
    v
Token-level FSM              (create_fsm_index_tokenizer)
    |
    v
Precomputed masks            (build_tokenizer_index)
+ longest-token coalescence
    |
    v
Constrained generation       (generate_constrained)
with GPT-2
```

Each module tackles one level of this pipeline. By the end, you will have built a system that guarantees LLM outputs match an arbitrary regex pattern, with significant performance optimizations over the naive approach.

2 Part 2: Meta-Generation [8 points]

When an LLM generates code, a single sample may or may not be correct. A natural idea is to generate **multiple samples** and then **select the best one**. But how do we decide which sample is "best" without already knowing the answer?

This part explores **meta-generation** strategies: generate n candidate solutions, rank them using different selection criteria, and pick the top-ranked one. We evaluate these strategies on the MBPP (Mostly Basic Programming Problems) benchmark using a locally-served LLM via Ollama.

Here is an example of a sample from MBPP:

```
{
    'task_id': 1,
    'text': 'Write a function to find the minimum cost path to reach
             (m, n) from (0, 0) for the given cost matrix cost[][]
             and a position (m, n) in cost[][].',
    'code': 'R = 3\r\nC = 3\r\ndef min_cost(cost, m, n): ...',
    'test_list': [
        'assert min_cost([[1, 2, 3], [4, 8, 2], [1, 5, 3]], 2, 2) == 8',
        'assert min_cost([[2, 3, 4], [5, 9, 3], [2, 6, 4]], 2, 2) == 12',
        'assert min_cost([[3, 4, 5], [6, 10, 4], [3, 7, 5]], 2, 2) == 16'],
    'test_setup_code': '',
    'challenge_test_list': []
}
```

From the `text` field, the LLM must generate Python code. For Strategy 3 below, the LLM is allowed to observe the execution of its code on the first test in `test_list`. To check whether the generated code is correct, we run all remaining tests.

2.1 Overview

Module	Description
`meta_generation.py`	Sample generation, ranking strategies, and evaluation
`utils.py`	Prompt construction, code extraction, and safe code execution (utility, no exercises)

2.2 The Problem: Best-of-N Selection

Given a coding problem, we:

1. Generate **n** candidate solutions from the LLM (with temperature > 0 for diversity)
2. **Rank** the candidates using a selection strategy
3. **Pick** the top-ranked candidate as our answer

The question is: which ranking strategy gives us the best chance of picking a correct solution?

2.3 Baseline: pass@1

The simplest metric is **pass@1**: what fraction of the n samples pass all tests? This measures the overall quality of the model but does not involve any selection — it is the expected success rate if we pick a sample at random.

```
pass@1 = (number of passing samples) / n
```

This serves as the baseline. Any selection strategy that consistently beats pass@1 is adding value.

2.4 Strategy 1: Best-of-N by Log-Probability (`rank_by_logprob`)

2.4.1 Idea

The LLM assigns a probability to each token it generates. We can use this as a **confidence score**: samples where the model was more confident (higher log-probabilities) might be more likely to be correct.

2.4.2 Algorithm

1. For each sample, compute the **average log-probability** across all generated tokens:

   ```
   avg_logprob = sum(logprob_i for each token i) / number_of_tokens
   ```

2. Rank samples by `avg_logprob` in descending order
3. Pick the top-ranked sample

2.4.3 Properties

- Fast: no execution needed, the log-probabilities come for free from the generation step
- Relies on the assumption that model confidence correlates with correctness
- Can fail when the model is confidently wrong

2.4.4 What to implement

- `_generate_one(prompt, model, **kwargs)`: generate a single code sample and extract its average log-probability from the API response.
- `rank_by_logprob(logprobs)`: sort sample indices by log-probability in descending order.

2.5 Strategy 2: MBR with Edit Similarity (`rank_by_edit_sim`)

2.5.1 Idea

Minimum Bayes Risk (MBR) decoding selects the sample that is most similar to all other samples. The intuition is: if many independent samples converge to similar code, that code is more likely to be correct (majority vote at the code level).

Here we measure similarity using **Levenshtein (edit) distance** between code strings.

2.5.2 Algorithm

1. For each pair of samples `(i, j)`, compute the normalized edit similarity:

   ```
   sim(i, j) = 1 - levenshtein_distance(code_i, code_j) / max(len(code_i), len(code_j))
   ```

2. For each sample `i`, compute its **gain** (total similarity to all other samples):

   ```
   gain(i) = sum(sim(i, j) for all j)
   ```

3. Rank samples by `gain` in descending order
4. Pick the top-ranked sample (the one most representative of the consensus)

2.5.3 Properties

- No execution needed — purely text-based
- Captures syntactic consensus across samples
- O(n^2) pairwise comparisons, but fast since it only compares strings
- Can be fooled by syntactically similar but semantically different code

2.5.4 What to implement

- `rank_by_edit_sim(codes)`: compute pairwise edit similarities, accumulate gains, and return sorted indices. (Hint: use the `jellyfish` library to compute Levenshtein distance.)

2.6 Strategy 3: MBR with Execution Similarity (`rank_by_exec_sim`)

2.6.1 Idea

Instead of comparing code as text, we compare code by its **behavior**: run each sample on a set of test inputs and compare the outputs. Two samples are similar if they produce the same results on the same inputs.

This is a stronger form of MBR — it captures semantic equivalence rather than syntactic similarity.

2.6.2 Algorithm

1. Extract function calls from the test cases (e.g., `func(arg1, arg2)` from `assert func(arg1, arg2) == expected`). (Hint: use `extract_func_calls` from `utils.py`.)
2. Execute each code sample with these function calls in sandboxed subprocesses. (Hint: use `execute_codes` from `utils.py`.)
3. For each pair `(i, j)`, compute the execution similarity:

   ```
   sim(i, j) = (number of calls where output_i == output_j) / (total number of calls)
   ```

   If either sample fails to execute, set `sim(i, j) = 0`.
4. Accumulate gains and rank as in Strategy 2.

2.6.3 Properties

- Most semantically meaningful: measures actual behavioral agreement
- Requires code execution (sandboxed for safety)
- More expensive than text-based methods, but more robust
- Can handle functionally equivalent but syntactically different code

2.6.4 What to implement

- `rank_by_exec_sim(codes, test_list)`: extract function calls, execute all samples, compute pairwise execution similarities, and return sorted indices. (Hint: use `execute_tests` from `utils.py`)

2.7 Evaluation (`evaluate_problem`)

For a single MBPP problem, the evaluation function:

1. Runs all n code samples against the full test suite to determine pass/fail for each
2. Applies each ranking strategy to select one sample
3. Reports whether the selected sample passes:
   - **pass@1**: average pass rate (no selection)

- **logprob**: does the top-by-logprob sample pass? (0 or 1)
- **edit**: does the top-by-edit-similarity sample pass? (0 or 1)
- **exec**: does the top-by-execution-similarity sample pass? (0 or 1)

2.7.1 What to implement

- `evaluate_problem(codes, logprobs, test_list)`: orchestrate all four evaluation methods and return scores.

2.8 Utility Module (`utils.py`)

This module provides infrastructure for prompt construction and safe code execution. It is adapted from bigcode-evaluation-harness and OpenAI's human-eval.

Key functions (provided, no exercises):

- `make_prompt(example, n_tests)`: build a zero-shot prompt from an MBPP example
- `extract_code(completion)`: extract Python code from markdown code blocks in LLM responses
- `extract_func_calls(test_list)`: parse function calls from assert statements
- `execute_tests(codes, tests)`: run each code sample against test cases in sandboxed subprocesses (parallel)
- `execute_codes(codes, calls)`: run each code sample with specific function calls and collect outputs (for execution similarity)
- `unsafe_execute(program, conn, timeout)`: isolated execution with timeout, I/O redirection, and `reliability_guard` to disable dangerous system calls

---

2.9 Running the Evaluation

2.9.1 Prerequisites

You need Ollama running locally with a code model. Pull the model first, then start the server:

```
ollama pull qwen2.5-coder:7b
OLLAMA_HOST=127.0.0.1:11435 ollama serve
```

For faster parallel inference, you can enable parallel requests and flash attention:

```
OLLAMA_HOST=127.0.0.1:11435 OLLAMA_NUM_PARALLEL=8 OLLAMA_FLASH_ATTENTION=1 ollama serve
```

2.9.2 Running

```
uv run python meta_generation/meta_generation.py
```

2.9.3 Command-line options

Flag	Default	Description
`--model`	`qwen2.5-coder:7b`	Ollama model to use
`--n`	`15`	Number of samples to generate
`--temperature`	`0.7`	Sampling temperature
`--top_p`	`0.95`	Top-p (nucleus) sampling
`--seed`	`1618`	Random seed for reproducibility
`--index`	`13`	MBPP test problem index
`--output`	`mbpp_eval_results.json`	Output file for results

2.9.4 Example output

```
Method       Score
--------------------
pass@1       0.6000
logprob      1.0000
edit         1.0000
exec         1.0000
```

This shows that while only 60% of individual samples pass, all three selection strategies successfully picked a correct solution. Save your result in the `mbpp_eval_results.json` output file.

2.10 Summary

```
MBPP problem
    |
    v
Generate n samples          (parallel, via Ollama)
    |
    v
Rank by 4 strategies:
  1. pass@1                  (baseline: average pass rate)
  2. logprob                 (model confidence)
  3. edit similarity         (textual consensus, MBR)
  4. execution similarity    (behavioral consensus, MBR)
    |
    v
Pick top sample per strategy
    |
    v
Evaluate against test suite
```

3 Part 3: Self-Correction [8 points]

Some tasks require **provably correct** code — code that is formally verified by an external tool. When the LLM's first attempt fails verification, can we use the error messages to iteratively fix it?

This part implements a **tree-search refinement pipeline** for generating formally verified Rust code using Verus, a verification tool for Rust. The LLM generates candidate implementations, Verus checks them, and error feedback drives iterative refinement until a fully verified program is found.

3.1 Overview

Module	Description
`self_correction.py`	Two-stage pipeline: parallel generation + tree-search refinement
`utils.py`	Verus integration, code validation, scoring, and error parsing
`examples/incr_list.rs`	Example Verus specification for the LLM to complete

3.2 Context: Formal Verification with Verus

Verus is a tool for proving that Rust programs satisfy formal specifications. A Verus program contains:

- **Preconditions** (`requires`): what must be true before the function runs
- **Postconditions** (`ensures`): what must be true after the function returns
- **Loop invariants** (`invariant`): what must hold at every iteration
- **Proof statements** (`assert`, proof blocks): hints to help the verifier

The LLM's task is to **complete the function body** (including any necessary proof statements and loop invariants) so that Verus can verify the postconditions hold.

3.2.1 Example specification (`examples/incr_list.rs`)

```
fn incr_list(l: Vec<i32>) -> (result: Vec<i32>)
    requires
        forall|i: int| 0 <= i < l.len() ==> l[i] + 1 <= i32::MAX,
    ensures
        result.len() == l.len(),
        forall|i: int| 0 <= i < l.len() ==> #[trigger] result[i] == l[i] + 1,
{
    // LLM must complete this function body
}
```

The function must increment every element by 1 and prove that the result satisfies both postconditions.

3.3 Stage 1: Parallel Initial Generation (`parallel_generator`)

3.3.1 Idea

Generate a large number of candidate implementations in parallel. Most will fail verification, but some may partially verify (some assertions pass, some fail). These partially-correct programs seed Stage 2.

3.3.2 Algorithm

1. We provide a prompt containing:
   - A system prompt with Verus expertise and worked examples
   - The specification (function signature + preconditions + postconditions)
   - Instructions to complete only the function body
2. Generate `first_stage` (default: 50) completions in parallel using a thread pool
3. For each generation, combine it with the specification and run Verus (`check`)
4. Classify results into three categories:
   - **Verified** (score = 1): fully verified, we are done
   - **Valid** (score = 0): partially verified (some assertions pass, some errors remain) — these proceed to Stage 2
   - **Invalid** (score = -1): rejected by validation checks or completely fails

3.3.3 What to implement

- `first_verif(generations, input_program, verus_path)`: for each generation, parse it with `utils.parse_generation`, run Verus with `utils.check`, and classify the result as verified, valid, or invalid.

3.4 Stage 2: Iterative Refinement with Tree Search

3.4.1 Idea

Take the partially-correct programs from Stage 1, feed Verus error messages back to the LLM, and ask it to fix the code. Repeat for multiple iterations, focusing search effort on the most promising branches.

3.4.2 The Refinement Loop

Each iteration performs five steps:

3.4.2.1 Step 1 — Expand For each current conversation state, generate one refinement by calling the LLM with the full conversation history (system prompt + original program + error feedback from previous iterations). The LLM sees the accumulated context and produces a corrected version.

```
State = [system_prompt, user_msg_with_program_and_errors, assistant_response,
         user_msg_with_new_errors, assistant_response, ...]
```

3.4.2.2 Step 2 — Score (Value Function) Evaluate each refined program with `evaluate_code`:

1. Run Verus on the code
2. Parse the output: count verified assertions and errors
3. Compute a score:
   - Base score = `num_verified / (num_verified + num_errors)`
   - Penalty: `-0.1` per parsed error, `-0.04` per note
   - Score = -1 if code is invalid (fails validation checks)
   - Score = 1 if fully verified (0 errors) and passes trivial-proof detection
4. Return `(score, error_message)`

3.4.2.3 Step 3 — Filter Remove all states with score < 0 (invalid code). If all states are invalid, retry the iteration.

3.4.2.4 Step 4 — Deduplicate Collapse states that produced identical code, keeping only the highest-scoring instance of each unique program. This avoids wasting search budget on redundant branches.

3.4.2.5 Step 5 — REBASE Sampling

REBASE (Regularized Best-first And Scoring Expansion) resamples the surviving states to maintain a fixed batch size while focusing on the most promising branches:

1. Convert scores to probabilities via softmax with temperature `tau`:

   ```
   P(i) = exp(score_i / tau) / sum_j exp(score_j / tau)
   ```

2. Sample `batch_size` states (with replacement) according to these probabilities
3. Append the error message to each sampled state for the next iteration

A low temperature (default: 0.1) makes the distribution sharp, concentrating effort on the highest-scoring states. Higher temperatures preserve more diversity.

3.4.2.6 Early Stopping

If any state achieves score >= 1 (fully verified), the loop exits immediately.

3.4.3 What to implement

- `filter_valid(new_states, scored)`: keep only states with non-negative scores, returning `(state, score, error)` triples.
- `deduplicate(valid_triples, input_program, iteration_number)`: collapse states that produced identical code, keeping the best score.
- `rebase_select(valid_triples, batch_size, rebase_temperature)`: sample `batch_size` states proportional to the softmax of their scores.
- `iterative_refinement(valid, input_program, verus_path, ...)`: the main loop orchestrating expand, score, filter, deduplicate, and REBASE steps. Uses the provided helper functions `initialize`, `refinement_message`, `expand_to_batch`, and `async_refinement_generator`.

3.5 Code Validation (`_validate_code`)

Before scoring, each program is checked against a set of rejection rules. Code is rejected (score = -1) if it:

- Contains `assume` statements (defeats the purpose of verification)
- Has a trivial postcondition (`ensures true`)
- Has more comments than actual code (>50% comments)
- Has no `ensures` clause
- Contains infinite loops (consecutive `{}`)
- Uses external verifiers (`#[verifier::external]` or `#[verifier::external_body]`)
- Has unreachable statements
- Uses `&mut` on non-`Vec` types

3.6 Trivial Proof Detection (`check_pairs`, `check_pairs_loop`)

A subtle failure mode: the LLM may produce code that Verus verifies, but only because the preconditions make the postconditions trivially true (e.g., contradictory preconditions). To detect this:

- **`check_pairs`**: inject `assert(false)` at the start of the function body. If Verus still reports 0 errors, the preconditions are contradictory and the proof is trivial.
- **`check_pairs_loop`**: inject `assert(false)` before closing braces of loop bodies to test whether loop invariants are trivially satisfied.

Programs that pass these tests are rejected as trivially verified.

3.7 Scoring Function (`evaluate_code`)

The scoring function serves as the value function for the tree search:

Condition	Score
Fails validation checks	-1
No verified assertions and no errors	-1
More than 20 verified assertions (suspicious)	-1
Trivially verified (`check_pairs` / `check_pairs_loop`)	-1
Fully verified (0 errors, passes trivial checks)	1
Partial: some verified, some errors	`num_verified / (num_verified + num_errors)` minus penalties

Error penalties are normalized: each error deducts `0.1 * (1 / total_assertions)` and each note deducts `0.04 * (1 / total_assertions)`.

3.8 System Prompt

The system prompt provides the LLM with Verus expertise:

- General notes on Verus syntax (loop invariants, spec vs. exec code, type conversions)
- Three worked examples of increasing complexity:
  1. `has_close_elements` — nested loops with overflow-safe arithmetic
  2. `rolling_max` — recursive spec functions with sequence operations
  3. `intersperse` — quantified specs with proof blocks and `assert_seqs_equal!`

3.9 Running the Pipeline

3.9.1 Prerequisites

- Verus installed and accessible, to install Verus
- Ollama running with a code model

```
ollama pull qwen2.5-coder:7b
```

3.9.2 Running

```
uv run python self_correction/self_correction.py --verus-path /path/to/verus
```

3.9.3 Command-line options

Flag	Default	Description
`--input`	`examples/incr_list.rs`	Verus specification file
`--model`	`qwen2.5-coder:7b`	LLM model
`--batch-size`	`32`	Parallel generation batch size
`--first-stage`	`8`	Number of initial programs
`--max-iters`	`5`	Maximum refinement iterations
`--temperature`	`0.7`	LLM sampling temperature
`--rebase-temperature`	`0.1`	Softmax temperature for REBASE
`--max-tokens`	`2048`	Max tokens per generation

Flag	Default	Description
`--base-url`	`http://localhost:11435/v1`	API endpoint
`--api-key`	`$OPENAI_API_KEY` or `ollama`	API key
`--verus-path`	`$VERUS_PATH`	Path to Verus binary
`--output`	`verified_program.rs`	Output file
`--valid-programs`	`None`	Path to pre-made valid programs (skips Stage 1)

In my experiments, `qwen2.5-coder:7b` can produce verified programs but requires a very large number of attempts. The pipeline also supports external API calls: with `gpt-4o`, verified programs were found using the default parameters above.

3.9.4 Bonus

Explore ways to improve the generation of verified programs. To earn extra credit, document your experiments in a markdown file: explain your approach and your results (failures can be as informative as successes!).

3.10 Summary: The Full Pipeline

```
Verus specification (requires + ensures)
    |
    v
Stage 1: Generate N candidates in parallel
    |
    v
Classify: verified / valid / invalid
    |
    v
Stage 2: Iterative refinement loop
    |
    +---> Expand:  LLM generates fix from error feedback
    |       |
    |       v
    |     Score:   Verus verifies, compute value function
    |       |
    |       v
    |     Filter:  Remove invalid states (score < 0)
    |       |
    |       v
    |     REBASE:  Softmax resampling toward best branches
    |       |
    |       v
    +--- Append error message, repeat
    |
    v
Output: verified program (or best attempt)
```